Decision Making in Bash using If-Else Constructs

Comparison Operators

Comparison operators are used to compare values in decision-making statements, usually within `if` statements. Here are the key comparison operators in Bash:

Operator	Description	Example
`-eq`	Equal to	`["$a" -eq "$b"]` checks if `a` is equal to `b`.
`-ne`	Not equal to	`["$a" -ne "$b"]` checks if `a` is not equal to `b`.
`-gt`	Greater than	`["$a" -gt "$b"]` checks if `a` is greater than `b`.
`-lt`	Less than	`["$a" -lt "$b"]` checks if `a` is less than `b`.
`-ge`	Greater than or equal to	`["$a" -ge "$b"]` checks if `a` is greater than or equal to `b`.
`-le`	Less than or equal to	`["$a" -le "$b"]` checks if `a` is less than or equal to `b`.

For String Comparisons:

- `==` : Checks if two strings are identical.
- `!=` : Checks if two strings are not identical.
- `-z` : Checks if the string is empty (`[ -z "$string" ]`).
- `-n` : Checks if the string is not empty (`[ -n "$string" ]`).

Logical Operators

Logical operators allow you to combine multiple conditions. In Bash, the main logical operators are:

Operator	Description	Example
`&&`	Logical AND	`["$a" -gt 10] && ["$a" -lt 20]` - checks if `a` is between 10 and 20.
`!`	Logical NOT	`[! -f "/path/to/file"]` - checks if the file does not exist.
`||`	Logical OR	`[! -f "$file1"] \|\| [! -f "$file2"];` checks if either file1 or file2 does not exist

Notes:

- When combining expressions, each condition should be enclosed in `[]` or `[[ ]]`.
- `&&` ensures both conditions are true, whereas `||` requires only one condition to be true.

Testing Conditions

In Bash, testing conditions are evaluated using either `test` or square brackets `[ ]`. These are generally used within `if` statements to check the truth of a condition.

Basic Syntax

```
if [ condition ]; #space is necessary after "if" and before and after
#condition inside the brackets
then
        Commands to execute if condition is true.
elif [ another condition ]; then
        Commands for alternate condition.
else
        Commands if all conditions fail.
fi to close the if structure.
```

Types of Testing

- **Arithmetic Tests**: Use comparison operators like `-eq`, `-ne`, etc., to compare integers.
- **String Tests**: Use `==`, `!=`, `-z`, and `-n` for string comparisons.
- **File Tests:** (explained further below).

Example

```
num=15
if [ "$num" -gt 10 ]; then
   echo "Greater than 10"
else
   echo "10 or less"
 fi
```

File Conditions

File conditions allow you to check properties of files and directories. They help determine if a file exists, if it's a directory, and if it has specific permissions.

Condition	Description	Example
`-e`	File exists	`[-e "$file"]`
`-f`	File is a regular file	`[-f "$file"]`
`-d`	Directory exists	`[-d "$dir"]`
`-r`	File is readable	`[-r "$file"]`
`-w`	File is writable	`[-w "$file"]`
`-x`	File is executable	`[-x "$file"]`
`-s`	File has non-zero size	`[-s "$file"]`

Example

```
   file="/path/to/file"
if [ -e "$file" ]; then
   echo "File exists"
   if [ -r "$file" ]; then
       echo "File is readable"
   else
       echo "File is not readable"
   fi
else
   echo "File does not exist"
 fi
```

Nested If Statements

Introduce examples where multiple conditions must be checked using nested `if` statements.

For Example:

```
if [ -e "$file" ]; then
if [ -r "$file" ]; then
echo "The file exists and is
readable."
Else
echo "The file exists but is
not readable."
fi
else
echo "The file does not exist."
fi
```

Exercise: Write a script that checks if a directory exists and whether it contains any readable files.

Using Logical Operators

Provide a task using `&&` and `||` operators for combined conditions. Example:

```
if [ -f "$file1" ] && [ -f "$file2" ]; then
echo "Both files exist."
elif [ -f "$file1" ] || [ -f "$file2" ]; then
echo "Only one of the files exists."
else
echo "Neither file exists."
fi
```

Exercise: Create a script that checks if either a `.txt` file or a `.log` file is present in a directory.

File Conditions with User Input

Develop a scenario where the script prompts the user for a file path and checks multiple properties of that file.

Exercise: Ask students to create a script that:

a) Prompts the user to enter a filename.
b) Checks if the file exists, is executable, and has non-zero size.
c) Displays appropriate messages based on each condition.

Advanced Decision-Making Scenarios

Challenge students to combine various condition types (e.g., arithmetic, string, and file tests) in a single script.

Exercise: Write a script to prompt for a directory path and then:

a) Check if the directory exists.
b) Check if it contains at least one file larger than 1KB.
c) Output the count of readable files in that directory.

Practice Scenario: Backup Script

Objective: Write a script that verifies if a backup file is created, its size is non-zero, and it has appropriate permissions before proceeding with further operations.

Solution Outline:

```
backup_file="/path/to/backup"
if [ -e "$backup_file" ] && [ -s "$backup_file" ] && [ -r "$backup_file" ] && [ -w "$backup_file" ]; then
echo "Backup file is ready for use."
else
echo "Backup file is missing or does not have the right properties."
fi
```

Case Study: User Account Checker

Objective: Write a script to check if a user account exists on the system.

Instructions:

a) Prompt the user to enter a username.
b) Check if the user exists using `getent passwd "$username"`.
c) Display appropriate messages based on whether the user exists.
d) If the user exists, check if the user has written access to a specific directory (e.g., `/home/$username`).

Sample Code:

```
echo "Enter the username:"
read username
if getent passwd "$username" > /dev/null 2>&1; then
    echo "User $username exists."
    if [-w "/home/$username"]; then
        echo "$username has written access to their home directory."
    else
        echo "$username does not have write access to their home directory."
    fi
else
    echo "User $username does not exist."
fi
```

Advanced Exercise: Disk Space Monitor

Objective: Write a script to monitor disk space usage and alert if usage exceeds a specified threshold.

Instructions:

a) Prompt the user to enter a disk space threshold percentage (e.g., 80%).
b) Check the root filesystem's usage (`df / | awk 'NR==2 {print $5}'`).
c) Display a warning if usage exceeds the threshold, and a message if it's below the threshold.

Exercise:

```
echo "Enter disk space threshold percentage (without %):"
read threshold
usage=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')
if ["$usage" -gt "$threshold"]; then
    echo "Warning: Disk usage is above $threshold%."
else
    echo "Disk usage is within limits."
 fi
```

Case Statement in Bash Scripting

A case statement in Bash scripting is a control flow structure that allows you to execute different code blocks based on the value of a variable or expression. It's similar to a `switch` statement in other programming languages.

Syntax:

```
case expression in
pattern1)
  commands
  ;;
pattern2)
  commands
  ;;
...
*)
  default commands
  ;;
esac
```

Explanation:

1. **`case expression in`**: This initiates the case statement, where `expression` is the variable or value to be matched.
2. **`pattern1) commands ;;`**: This is a case pattern. If `expression` matches `pattern1`, the following `commands` are executed. The `;;` signifies the end of the case block.
3. **`*) default commands ;;`**: This is the default case. If `expression` doesn't match any of the preceding patterns, the `default commands` are executed.

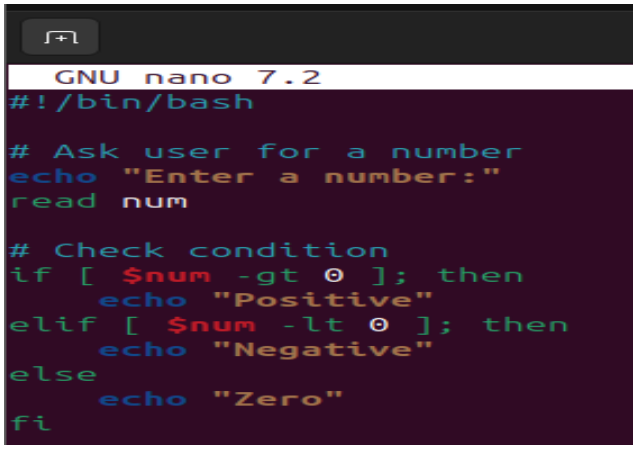
Example:

```
#!/bin/bash
echo "Enter a fruit name:"
read fruit
case $fruit in
  "apple")
    echo "Apples are red."
    ;;
"banana")
    echo "Bananas are yellow."
    ;;
  "orange")
    echo "Oranges are orange."
    ;;
  *)
    echo "I don't know that fruit."
    ;;
esac
```

Practice Questions

Basic Questions:

1) **Simple Comparison:** Write a script that prompts the user for a number and prints "Positive" if the number is greater than 0, "Negative" if it's less than 0, and "Zero" otherwise.



```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano check_number.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x check_number.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./check-number.sh
bash: ./check-number.sh: No such file or directory
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./check_number.sh
Enter a number:
15
Positive
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./check_number.sh
Enter a number:
-20
Negative
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

2) **File Existence and Permissions:** Write a script that checks if a file named "myfile.txt" exists in the current directory. If it exists, check if it's readable and writable. Print appropriate messages.

```
GNU nano 7.2
#!/bin/bash

# Check if file exists
if [ -e "myfile.txt" ]; then
    echo "File exists"

    # Check readable
    if [ -r "myfile.txt" ]; then
        echo "File is readable"
    else
        echo "File is NOT readable"
    fi

    # Check writable
    if [ -w "myfile.txt" ]; then
        echo "File is writable"
    else
        echo "File is NOT writable"
    fi
else
    echo "File does not exist"
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano file_check.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x file_check.sh
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./file_check.sh
File does not exist
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

3) **String Comparison:** Write a script that asks the user for their name. If the name is "Alice," print a special message. Otherwise, print a generic greeting.

```
  GNU nano 7.2
#!/bin/bash

echo "Enter your name:"
read name

if [ "$name" = "Alice" ]; then
    echo "Welcome Alice! You are special."
else
    echo "Hello $name!"
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano name_check.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x name_check.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./name_check.sh
Enter your name:
Abdullah
Hello Abdullah!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

Intermediate Questions:

4) **Multiple Conditions:** Write a script that checks if a given year is a leap year. A year is a leap year if it is divisible by 4, but not by 100, unless it is also divisible by 400.

```
  GNU nano 7.2
#!/bin/bash

echo "Enter a year:"
read year

if (( (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0) )); then
    echo "Leap Year"
else
    echo "Not a Leap Year"
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano leap_year.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x leap_year.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./leap_year.sh
Enter a year:
2026
Not a Leap Year
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

5) **Nested If-Else:** Write a script that checks if a number is positive, negative, or zero. If it's positive, check if it's even or odd.

```
  GNU nano 7.2
#!/bin/bash

echo "Enter a number:"
read num

if [ $num -gt 0 ]; then
    echo "Positive"

    if [ $((num % 2)) -eq 0 ]; then
        echo "Even"
    else
        echo "Odd"
    fi

elif [ $num -lt 0 ]; then
    echo "Negative"
else
    echo "Zero"
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano number_type.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x number_type.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./number_type.sh
Enter a number:
2000
Positive
Even
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

6) **Case Statement:** Write a script that prompts the user for an operation (+, -, *, /) and two numbers. Use a case statement to perform the specified operation and print the result.

```
  GNU nano 7.2
#!/bin/bash

echo "Enter first number:"
read a

echo "Enter second number:"
read b

echo "Enter operation (+, -, *, /):"
read op

case $op in
    +) echo "Result = $((a + b))" ;;
    -) echo "Result = $((a - b))" ;;
    \*) echo "Result = $((a * b))" ;;
    /)
        if [ $b -eq 0 ]; then
            echo "Error: Division by zero"
        else
            echo "Result = $((a / b))"
        fi
        ;;
    *) echo "Invalid operation"
esac
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano calculator.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x calculator.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./calculator.sh
Enter first number:
1600
Enter second number:
290
Enter operation (+, -, *, /):
+
Result = 1890
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

Advanced Questions:

7) **Function with Decision-Making:** Write a function that takes a filename as input and returns 1 if the file exists and is readable, 0 otherwise. Use this function in a script to check multiple files.

```
  GNU nano 7.2
#!/bin/bash

# Function definition
check_file() {
    if [ -r "$1" ]; then
        return 1
    else
        return 0
    fi
}

# Check multiple files
for file in "$@"
do
    check_file "$file"
    if [ $? -eq 1 ]; then
        echo "$file is readable and exists"
    else
        echo "$file does NOT exist or is not readable"
    fi
done
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano file_function.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x file_function.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./file_function.sh
```

8) **Error Handling:** Write a script that attempts to divide two numbers. Use an if-else statement to check for division by zero and print an error message if it occurs.

```
  GNU nano 7.2
#!/bin/bash

echo "Enter first number:"
read a

echo "Enter second number:"
read b

if [ $b -eq 0 ]; then
    echo "Error: Cannot divide by zero"
else
    result=$((a / b))
    echo "Result = $result"
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano divide.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x divide.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./divide.sh
Enter first number:
50
Enter second number:
5
Result = 10
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./divide.sh
Enter first number:
200
Enter second number:
6
Result = 33
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

9) **Complex File Operations:** Write a script that checks if a directory exists. If it does, check if it contains any files with the ".txt" extension. If it does, print the names of the files.

```
  GNU nano 7.2
#!/bin/bash

echo "Enter directory name:"
read dir

if [ -d "$dir" ]; then
    echo "Directory exists"

    txt_files=$(ls "$dir"/*.txt 2>/dev/null)

    if [ -z "$txt_files" ]; then
        echo "No .txt files found"
    else
        echo "Text files found:"
        ls "$dir"/*.txt
    fi
else
    echo "Directory does not exist"
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano dir_check.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x dir_check.sh
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./dir_check.sh
Enter directory name:
demo1.sh
Directory does not exist
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./dir_check.sh
Enter directory name:
bash_lab
Directory exists
Text files found:
bash_lab/log.txt  bash_lab/textfile.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

10) **System Monitoring:** Write a script that checks the CPU usage and disk space usage. If either exceeds a certain threshold, send an email alert.

```
abdullah-sohail@abdullah-sohail-VMw
 GNU nano 7.2                                                system_monitc
#!/bin/bash

# Set thresholds
CPU_THRESHOLD=80
DISK_THRESHOLD=80

# Get CPU usage
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')

# Get disk usage
DISK_USAGE=$(df / | tail -1 | awk '{print $5}' | sed 's/%//')

echo "CPU Usage: $CPU_USAGE%"
echo "Disk Usage: $DISK_USAGE%"

# Check thresholds
if (( $(echo "$CPU_USAGE > $CPU_THRESHOLD" | bc -l) )) || [ $DISK_USAGE -gt $DISK_THRESHOLD ]; then
    echo "ALERT: High system usage!"

    # Send email (modify email address)
    echo "System usage exceeded limits" | mail -s "System Alert" your_email@example.com
fi
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano system_monitor.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x system_monitor.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./system_monitor.sh
CPU Usage: 8.9%
Disk Usage: 39%
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```